

PROJET MINISHELL

Objectif de la fonction

Cette fonction `copy_env` permet de copier les variables d'environnement (`envp`) dans une liste chaînée (`t_env_var`). Elle transforme le tableau de chaînes de caractères fourni par le shell (`char **envp`) en une structure de liste plus facile à manipuler.

Explication ligne par ligne

```
t_env_var    *copy_env(char **envp, t_shell *shell)
```

Déclaration de la fonction :

`char **envp` : Un tableau de chaînes contenant les variables d'environnement (chaque élément est sous la forme "VAR=VALEUR").

`t_shell *shell` : Un pointeur vers la structure `t_shell`, utilisé pour gérer les erreurs et allouer de la mémoire de façon sécurisée.

```
t_env_var    *env_list;
t_env_var    *new_var;
char         *name;
char         *value;
int          i;
```

Déclaration des variables locales :

`env_list` : La tête de la liste chaînée qui contiendra les variables d'environnement.

`new_var` : Une variable temporaire pour stocker un nouvel élément de la liste.

`name` : Contient le nom de la variable d'environnement (avant =).

`value` : Contient la valeur de la variable d'environnement (après =).

`i` : Compteur pour parcourir `envp`.

```
env_list = NULL;
i = 0;
```

Initialisation :

`env_list = NULL;` : La liste est vide au départ.

`i = 0;` : On commence à parcourir `envp` depuis le premier élément.

```
while (envp && envp[i])
```

Boucle pour parcourir `envp`, `i` représente une ligne du tableau:

Imaginons `envp[]` :

```
envp[0] = "USER=root"           `=` trouvé → on ajoute "USER=root" à la
liste
envp[1] = "HOME=/home/root"     `=` trouvé → on ajoute "HOME=/home/root"
envp[2] = "PATH=/bin:/usr/bin"  `=` trouvé → on ajoute
"PATH=/bin:/usr/bin"
envp[3] = "INVALIDVAR"          PAS DE `=` → `continue` ignore cette ligne
```

`envp` doit être non nul (sécurité).

`envp[i]` doit exister (on s'arrête à la fin du tableau).

```
char *eq = ft_strchr(envp[i], '=');
```

Recherche du `=` dans `envp[i]` :

`ft_strchr(envp[i], '=')` retourne un pointeur vers la première occurrence de `=` dans la chaîne.

Si envp[i] est "USER=root", alors eq pointe vers = dans "USER=root".

```
if (!eq)
    continue;
```

Gestion des erreurs :

Si envp[i] ne contient pas de =, on ne peut pas diviser en name=value donc on ignore cette ligne et on passe à la suivante (continue saute le reste du code et recommence while à l'itération suivante).

```
name = ft_strndup(envp[i], eq - envp[i]);
```

Extraction du nom de la variable :

ft_strndup(envp[i], eq - envp[i]) crée une copie des caractères avant =.
eq - envp[i] calcule la longueur de la partie "USER" dans "USER=root", pour dupliquer "USER".

```
value = safe_strdup(eq + 1, shell);
```

Extraction de la valeur :

eq + 1 pointe vers le premier caractère après =.
safe_strdup(eq + 1, shell) duplique cette partie pour obtenir "root" dans "USER=root".

```
if (!name || !value)
    return (NULL);
```

Sécurité en cas d'échec de l'allocation mémoire :

Si ft_strndup ou safe_strdup a échoué (NULL), on retourne immédiatement NULL.

```
new_var = env_new_var(name, value, shell);
```

Création d'un nouvel élément de liste chaînée :

env_new_var(name, value, shell) crée une nouvelle structure t_env_var contenant "USER" et "root".

```
env_add_var(&env_list, new_var);
```

Ajout de la nouvelle variable dans la liste :

env_add_var(&env_list, new_var) ajoute le nouvel élément new_var au début de la liste env_list.

```
free(name);
free(value);
```

Libération des chaînes temporaires :

name et value ont été copiés dans env_new_var(), donc on les libère pour éviter les fuites mémoire.

```
i++;
```

Passage à la variable suivante dans envp

On continue à la ligne suivante de envp.

```
return (env_list);
```

Retourne la liste chaînée avec toutes les variables copiées.

Résumé

Que fait cette fonction ?

Parcourt envp (tableau des variables d'environnement).
Divise chaque élément en name=value (avant et après =).
Crée une structure t_env_var pour stocker name et value.
Ajoute cette variable dans une liste chaînée.
Retourne la liste chaînée complète.

Pourquoi continue quand = est absent ?

Si envp[i] est mal formé ("INVALIDVAR" au lieu de "VAR=VALEUR"), on l'ignore et on passe à l'élément suivant.

Exemple d'exécution

Cas d'entrée :

```
char *envp[] = {"USER=root", "HOME=/home/root", "PATH=/bin:/usr/bin", NULL};
```

Après exécution de copy_env(envp, shell)

Liste chaînée résultante :

```
-> name: "USER", value: "root"  
-> name: "HOME", value: "/home/root"  
-> name: "PATH", value: "/bin:/usr/bin"
```

Conclusion

Cette fonction est essentielle car elle transforme le tableau envp en une structure modifiable et gérable dans minishell. Cela permet de modifier les variables (export, unset, etc.).

Notes

Optimisation possible : Vérifier NULL avant free(name) et free(value).

Bonne gestion des erreurs avec safe_strdup et return NULL en cas d'échec mémoire.

Facile à comprendre grâce à sa structure claire.

```
*****  
*****
```

Fonction : init_shell()

```
void init_shell(t_shell *shell, char **envp)
```

shell : Pointeur vers la structure t_shell qui stocke l'état du Minishell.

envp : Tableau contenant l'environnement du système (char **).

Le but de cette fonction :

Copier envp dans shell->env sous forme de liste chaînée.
Vérifier que la copie a réussi.
Initialiser les valeurs par défaut (exit_stat, running).
Remplacer la valeur de OLDPWD par une chaîne vide.

Copier l'environnement (envp) dans shell->env

```
shell->env = copy_env(envp, shell);
```

copy_env() transforme envp (un tableau char **) en une liste chaînée t_env_var.

À ce stade :

shell->env contient une copie malloc des variables d'environnement.

Vérifier que la copie a réussi

```
if (!shell->env)
{
    perror("Failed to initialize environment");
    cleanup_shell(shell, NULL, NULL, NULL);
    exit(EXIT_FAILURE);
}
```

Que se passe-t-il si copy_env() échoue ?

shell->env == NULL Erreur !
On affiche un message d'erreur avec perror().
On nettoie le shell avec cleanup_shell().
On quitte le programme avec exit(EXIT_FAILURE);.

Pourquoi cleanup_shell() ici ?

Si une partie de shell->env a été allouée, on évite les fuites mémoire avant de quitter.

Remplacer OLDPWD par une chaîne vide

```
set_env_var(&shell->env, "OLDPWD", "", shell);
```

Pourquoi faire ça ?

Dans Bash, OLDPWD est vide au démarrage (pas supprimé).
Avant, on faisait :

```
unset_env_var(&shell->env, "OLDPWD");
```

Mais ça supprimait OLDPWD, alors que Bash le garde vide.

Donc maintenant, on met OLDPWD="".

Initialiser les valeurs par défaut

```
shell->exit_stat = 0; // Initialisation du statut de sortie
shell->running = 1;  // Le shell est actif
```

À quoi servent ces valeurs ?

exit_stat = 0 → Stocke le dernier code de sortie (echo \$? l'affichera).
running = 1 → Indique que le shell est actif (contrôle la boucle while).

Résumé (ligne par ligne)

Ligne Explication

shell->env = copy_env(envp, shell); Copie envp dans shell->env sous forme de liste chaînée.

if (!shell->env) { perror(); cleanup_shell(); exit(); } Vérifie si la copie a échoué → Nettoie et quitte le shell.

set_env_var(&shell->env, "OLDPWD", "", shell); Remplace OLDPWD par une chaîne vide (plutôt que le supprimer).

shell->exit_stat = 0; Initialise le code de sortie (echo \$?).

shell->running = 1; Active le shell (permet d'entrer dans la boucle while).

Conclusion

Cette fonction prépare shell avant de démarrer minishell.

Elle copie l'environnement,
Gère les erreurs si besoin,
Initialise OLDPWD,
Définit les valeurs de base (exit_stat et running).

Analyse détaillée de read_user_input()

But de la fonction :

Cette fonction lit l'entrée utilisateur et retourne la commande entrée sous forme d'une chaîne de caractères.

Explication ligne par ligne :

```
char *read_user_input(t_shell *shell)
```

La fonction retourne un char *, qui est la ligne de commande entrée par l'utilisateur.

Elle prend en paramètre un t_shell *shell, pour accéder aux variables du shell comme exit_stat et running.

```
char *prompt;  
char *input;
```

Déclare deux pointeurs char * :

prompt : contient le texte du prompt (minishell> en couleur).
input : stocke la ligne de commande saisie par l'utilisateur.

```
prompt = get_prompt(shell);
```

Appelle get_prompt(shell) pour générer le prompt dynamiquement, en incluant la couleur selon exit_stat.

```
if (!prompt)  
{  
    perror("[ERROR] Failed to generate prompt");  
    return (NULL);  
}
```

Gestion d'erreur :

Si get_prompt(shell) échoue et retourne NULL, on affiche une erreur avec perror().

La fonction retourne NULL pour signaler un problème.

```
input = readline(prompt);
```

Attend une entrée de l'utilisateur via readline(prompt), qui :

Affiche prompt.
Attend une saisie clavier.
Retourne la ligne entrée sous forme d'une chaîne allouée dynamiquement.
Retourne NULL si l'utilisateur appuie sur Ctrl+D.

Analyse détaillée de readline(prompt);

But de la fonction readline(prompt); :

Affiche le prompt (prompt est la chaîne affichée avant la saisie).
Attend une saisie utilisateur (l'utilisateur tape une commande et appuie sur Entrée).
Retourne la saisie sous forme d'une chaîne de caractères allouée dynamiquement.

Explication de readline(prompt);

input = readline(prompt);

readline() appartient à la bibliothèque readline.h (GNU Readline).
Elle prend un char *prompt en paramètre, qui est la chaîne affichée avant la saisie.
Elle :

Affiche le prompt à l'écran (minishell> en couleur dans notre cas).
Attend que l'utilisateur tape une ligne et appuie sur Entrée.
Retourne la ligne entrée sous forme d'une chaîne allouée dynamiquement (par malloc()).
Si l'utilisateur appuie sur Ctrl+D, readline() retourne NULL.

Exemple :

Affichage : minishell> (attend l'entrée de l'utilisateur)
L'utilisateur tape : ls -l
Après avoir appuyé sur Entrée, input contient "ls -l" (chaîne allouée dynamiquement).

Pourquoi prompt est un argument de readline() ?

readline(prompt) permet d'afficher un texte avant la saisie.

Exemple concret :

input = readline("Entrez une commande: ");

Affichera Entrez une commande: avant d'attendre l'entrée utilisateur.

Dans Minishell, prompt n'est pas une simple chaîne, mais une chaîne formatée avec une couleur dynamique :

prompt = get_prompt(shell);

La couleur change en fonction du dernier exit_stat, donc on ne peut pas juste afficher "minishell> " en dur.

prompt est généré dynamiquement avec ft_strjoin_three().
Pourquoi free(prompt); juste après readline(prompt); ?

Problème :
prompt est généré dynamiquement via malloc(), donc il occupe de la mémoire.

D'où vient prompt ?

prompt = get_prompt(shell); // Alloué dynamiquement

Dans get_prompt(), on a :

final_prompt = ft_strjoin_three(color, base_prompt, "\001" COLOR_RESET "\002", shell);

ft_strjoin_three() alloue une nouvelle chaîne avec safe_malloc().
Après avoir affiché le prompt avec readline(prompt);, on n'en a plus besoin.
Conséquence :

```
free(prompt);
```

Libère la mémoire allouée pour éviter une fuite mémoire.

Ne touche pas `input` (car `input` est une nouvelle chaîne allouée par `getline()`, différente de `prompt`).

Résumé simplifié

```
getline(prompt);
```

Affiche le prompt avant la saisie.

Attend que l'utilisateur tape une commande et appuie sur Entrée.

Retourne la commande sous forme d'une chaîne allouée dynamiquement.

Pourquoi `prompt` en paramètre ?

`prompt` définit le texte affiché avant la saisie.

Il est construit dynamiquement (`get_prompt()`).

Pourquoi `free(prompt);` après `getline(prompt);` ?

`prompt` est alloué dynamiquement (via `malloc()`).

Après l'appel à `getline()`, on n'en a plus besoin, donc on le libère.

Exemple détaillé avec valeurs concrètes :

Avant `getline()` :

```
prompt = "\033[32minishell> \033[0m"; // (GREEN "minishell> " RESET)
```

L'écran affiche :

```
minishell>
```

Après `getline(prompt);` :

L'utilisateur tape `echo hello` et appuie sur Entrée.

`getline(prompt);` retourne `"echo hello"` (chaîne allouée dynamiquement).

`input` pointe vers `"echo hello"`.

```
free(prompt);
```

On libère `prompt` car on n'en a plus besoin.

État mémoire après `getline(prompt);`

Variable	Contenu	Type d'allocation
----------	---------	-------------------

<code>prompt</code>	<code>\033[32minishell> \033[0m</code>	<code>malloc()</code> (libéré)
---------------------	---	--------------------------------

<code>input</code>	<code>"echo hello"</code>	<code>malloc()</code> (utilisé)
--------------------	---------------------------	---------------------------------

Conclusion

`getline(prompt);` affiche le prompt et attend une entrée utilisateur.

`prompt` est une chaîne allouée dynamiquement, donc on doit le libérer après usage (`free(prompt);`).

`input` est une nouvelle allocation mémoire retournée par `getline()`, qu'on utilisera ensuite.

```
free(prompt);
```

Libère la mémoire allouée pour le prompt, car `get_prompt()` utilise `malloc()`.

```
if (!input) // Gestion de Ctrl+D
```

```
{
```

```
    printf("exit\n");
```

```
    shell->running = 0;
```

```
    return (NULL);
```

```
}
```

Cas où l'utilisateur appuie sur Ctrl+D (EOF) :

Affiche "exit" (comme Bash).
Met shell->running = 0 pour arrêter la boucle principale.
Retourne NULL pour signaler la fin du shell.

```
if (*input) // Si l'entrée n'est pas vide
    add_history(input);
```

Ajoute la commande à l'historique, si elle n'est pas vide.

add_history(input) permet d'utiliser ↑ et ↓ pour rappeler les anciennes commandes.

```
return (input);
```

Retourne la ligne de commande entrée, qui sera traitée dans process_input().
Résumé de read_user_input()

Affiche le prompt avec get_prompt().
Attend une commande utilisateur avec readline().
Gère Ctrl+D en fermant proprement le shell.
Ajoute la commande à l'historique si elle n'est pas vide.
Retourne la commande entrée.

```
*****
*****
```

D'accord, je vais maintenant détailler chaque ligne et expliquer pourquoi elles font ce qu'elles font, pas seulement ce qu'elles font. On commence par get_prompt_color() puis get_prompt().

Analyse ultra-détaillée de get_prompt_color()
But de la fonction :

Cette fonction détermine la couleur du prompt en fonction du statut de sortie (exit_stat) de la dernière commande.

Décryptage ligne par ligne
Déclaration de la fonction

```
const char *get_prompt_color(int last_exit_status)
```

Explication technique :

Retourne un const char *, c'est-à-dire une chaîne de caractères constante (le code couleur).

Prend un int en argument (last_exit_status), qui correspond à l'état de sortie de la dernière commande exécutée.

Condition principale

```
if (last_exit_status == 0)
    return ("\001" GREEN "\002");
```

Pourquoi last_exit_status == 0 ?

Dans les shells Unix, une commande réussie retourne 0 comme code de sortie.
Exemple : si on tape ls et qu'il s'exécute correctement, alors \$? (le dernier code de sortie) vaut 0.

Que fait cette ligne ?

Si last_exit_status == 0, on retourne "\001" GREEN "\002".

GREEN est une macro définie (probablement "\033[32m" qui correspond à la couleur verte en ANSI).

\001 et \002 sont des caractères d'échappement spéciaux pour readline :

\001 : Début du code couleur.

\002 : Fin du code couleur.

Pourquoi \001 et \002 ?

readline() interprète ces caractères comme une séquence non visible.

Sans eux, le texte du prompt pourrait être mal aligné.

Concrètement, cette ligne retourne une séquence ANSI de couleur verte qui n'affecte pas le comptage des caractères.

Sinon, on retourne le rouge

```
return ("\001" RED "\002");
```

Pourquoi pas de else ?

Si le premier if n'est pas validé, alors on arrive automatiquement ici.

Pas besoin d'un else, car cette ligne est exécutée uniquement si last_exit_status != 0.

Pourquoi RED ?

RED est aussi une macro définie ("\033[31m"), qui met le texte en rouge.

En résumé, cette ligne fait en sorte que si la dernière commande a échoué (exit_stat != 0), alors on affiche un prompt en rouge.

Résumé ultra-simplifié

Statut de la dernière commande (exit_stat)	Valeur retournée	Couleur du prompt
--	------------------	-------------------

0 (succès)	"\001" GREEN "\002"	Vert
------------	---------------------	------

≠ 0 (échec)	"\001" RED "\002"	Rouge
-------------	-------------------	-------

Analyse ultra-détaillée de get_prompt()

But de la fonction :

Elle génère dynamiquement la chaîne du prompt, en combinant la couleur, le texte minishell>, et la réinitialisation des couleurs.

Décryptage ligne par ligne

Déclaration de la fonction

```
char *get_prompt(t_shell *shell)
```

Retourne un char * (le prompt final, alloué dynamiquement).

Prend un t_shell *shell en argument pour accéder au exit_stat et déterminer la couleur.

Déclaration des variables

```
const char *color;
```

```
const char *base_prompt;
```

```
char *final_prompt;
```

color : Stockera le code couleur ("\001\033[32m\002" pour vert ou "\001\033[31m\002" pour rouge).

base_prompt : Contiendra la chaîne "minishell> ".

final_prompt : Stockera le résultat final après concaténation.

Récupération de la couleur et du texte

```
color = get_prompt_color(shell->exit_stat);
```

```
base_prompt = "minishell> ";
```

Pourquoi on appelle get_prompt_color() ?

Pour déterminer la couleur en fonction du dernier `exit_stat`.

Pourquoi `base_prompt = "minishell> "`; en dur ?

Car le texte du prompt est fixe (`minishell>`) et ne change pas, seule la couleur change.

Vérification de `color` et `base_prompt`

```
if (!color || !base_prompt)
{
    print_error("get_prompt", "Invalid color or base_prompt");
    return (NULL);
}
```

Pourquoi cette vérification ?

On s'assure que `color` et `base_prompt` ne sont pas `NULL` avant de les utiliser.
En théorie, ce cas ne devrait jamais arriver, mais c'est une précaution contre un bug imprévu.

Que fait `print_error()` ?

Affiche un message d'erreur si un problème est détecté.

Construction du prompt final

```
final_prompt = ft_strjoin_three(color, base_prompt, "\001" COLOR_RESET "\002",
shell);
```

Pourquoi `ft_strjoin_three()` ?

Cette fonction concatène proprement `color`, `base_prompt` et `COLOR_RESET` (`\033[0m` pour réinitialiser la couleur).

Résultat :

"color + base_prompt + reset"
Exemple avec `exit_stat == 0` (succès) :

```
"\001\033[32m\002" + "minishell> " + "\001\033[0m\002"
```

Ce qui s'affiche en vert :

```
minishell>
```

Pourquoi `COLOR_RESET` ?

Pour éviter que la couleur du texte persiste après le prompt.

Vérification de `final_prompt`

```
if (!final_prompt)
{
    print_error("get_prompt", "Failed to allocate memory for prompt");
    return (NULL);
}
```

Pourquoi vérifier `final_prompt` ?

`ft_strjoin_three()` utilise `safe_malloc()`, qui peut échouer.
On évite un crash en affichant un message d'erreur et en retournant `NULL`.

Résumé ultra-simplifié

Détermine la couleur avec `get_prompt_color()`.
Définit le texte fixe ("minishell> ").
Concatène le tout avec `ft_strjoin_three()`.
Retourne le prompt final, alloué dynamiquement.

Exemple complet du comportement

exit_stat	color (renvoyé par <code>get_prompt_color</code>)	final_prompt (concaténé)
0 (succès)	"\001\033[32m\002" (Vert)	"minishell> " en vert
1 (échec)	"\001\033[31m\002" (Rouge)	"minishell> " en rouge

Conclusion

`get_prompt_color()` détermine la couleur du prompt en fonction du dernier code de sortie.
`get_prompt()` crée dynamiquement la chaîne complète, avec couleur et reset.
On libère toujours prompt après `readline()` pour éviter les fuites mémoire.

D'accord, je vais analyser chaque ligne en détail pour les fonctions suivantes :

`ft_strjoin_three()` → Concatène trois chaînes.
`ft_strjoin_multiple()` → Concatène plusieurs chaînes.
`array_len()` → Calcule la longueur d'un tableau de chaînes.
`free_array()` → Libère un tableau de chaînes.

Analyse ultra-détaillée de `ft_strjoin_three()`
But de la fonction :

Concatène trois chaînes (`s1`, `s2`, `s3`) dans une nouvelle chaîne allouée dynamiquement.
Décryptage ligne par ligne
Déclaration de la fonction

```
char *ft_strjoin_three(const char *s1, const char *s2, const char *s3, t_shell *shell)
```

Retourne un `char *` (une nouvelle chaîne allouée dynamiquement).
Prend en entrée :

`s1`, `s2`, `s3` → Les trois chaînes à concaténer.
`shell` → Utilisé pour gérer l'allocation mémoire via `safe_malloc()`.

Vérification des entrées

```
if (!s1 || !s2 || !s3)
{
    print_error("ft_strjoin_three", "NULL argument provided");
    return (NULL);
}
```

Pourquoi vérifier `s1`, `s2`, et `s3` ?

Si l'un des trois est `NULL`, on ne peut pas les concaténer, donc on affiche une erreur et on retourne `NULL`.
On évite un segfault en cas de mauvais appel.

Calcul de la longueur totale

```
len = ft_strlen(s1) + ft_strlen(s2) + ft_strlen(s3) + 1;
```

Pourquoi +1 ?

On ajoute 1 pour prendre en compte le \0 de fin de chaîne.

Allocation mémoire

```
result = safe_malloc(len, shell);
if (!result)
    return (NULL);
```

Pourquoi safe_malloc(len, shell); ?

On alloue un espace mémoire de la taille exacte de la future chaîne.
Si malloc échoue, safe_malloc() retourne NULL et affiche une erreur.

Initialisation de result à 0

```
ft_memset(result, 0, len);
```

Pourquoi ft_memset() ?

Pour remplir la mémoire avec des \0 et éviter des valeurs aléatoires.

Copie et concaténation des chaînes

```
ft_strlcpy(result, s1, len);
ft_strlcat(result, s2, len);
ft_strlcat(result, s3, len);
```

Pourquoi ft_strlcpy() et ft_strlcat() ?

ft_strlcpy(result, s1, len); → Copie s1 dans result.
ft_strlcat(result, s2, len); → Ajoute s2 à la suite de s1.
ft_strlcat(result, s3, len); → Ajoute s3 après s2.

Retour du résultat final

```
return (result);
```

On retourne la chaîne nouvellement allouée contenant s1 + s2 + s3.
Résumé ultra-simplifié

Vérifie que s1, s2, s3 existent.
Calcule la longueur totale + allocation mémoire.
Copie s1, concatène s2, puis s3.
Retourne la nouvelle chaîne.

Analyse ultra-détaillée de ft_strjoin_multiple()
But de la fonction :

Concatène un nombre variable de chaînes.
Décryptage ligne par ligne
Déclaration de la fonction

```
char *ft_strjoin_multiple(t_shell *shell, int count, ...)
```

Retourne un char * (une nouvelle chaîne allouée dynamiquement).
Prend en entrée :

count → Nombre de chaînes à concaténer.
... → Liste de chaînes en arguments variables (va_list).
shell → Utilisé pour safe_malloc().

Déclaration et initialisation

```
va_list    args;
```

```
char *result;
char *temp;
size_t total_len;
int i;
```

Pourquoi `va_list args; ?`

Permet de récupérer un nombre variable d'arguments.

Pourquoi `total_len = 1; ?`

On prévient une erreur en s'assurant que `result` aura au moins 1 caractère (`\0`).

Calcul de la longueur totale

```
va_start(args, count);
i = 0;
while (i < count)
{
    const char *current;

    current = va_arg(args, const char *);
    if (current)
        total_len += ft_strlen(current);
    i++;
}
va_end(args);
```

Pourquoi `va_start(args, count); ?`

Démarre la lecture des arguments variables (...).

Pourquoi `va_arg(args, const char *); ?`

Récupère chaque chaîne passée en argument.

Pourquoi `ft_strlen(current); ?`

Ajoute la longueur de chaque chaîne à `total_len`.

Allocation de mémoire

```
result = safe_malloc(total_len, shell);
if (!result)
    return (NULL);
*result = '\0';
```

Pourquoi `safe_malloc(total_len, shell); ?`

Alloue l'espace nécessaire pour toutes les chaînes concaténées.
Si `malloc` échoue, affiche une erreur et retourne `NULL`.

Pourquoi `*result = '\0'; ?`

Pour éviter d'avoir un pointeur non-initialisé.

Concaténation des chaînes

```
va_start(args, count);
i = 0;
while (i < count)
{
    temp = va_arg(args, char *);
```

```

    if (!temp)
        temp = "";
    ft_strlcat(result, temp, total_len);
    i++;
}
va_end(args);

```

Pourquoi va_arg() à nouveau ?

On relit la liste des arguments après avoir calculé la taille.

Pourquoi if (!temp) temp = "" ; ?

Si une chaîne est NULL, on la remplace par une chaîne vide.

Pourquoi ft_strlcat() ?

Pour ajouter chaque chaîne l'une après l'autre.

Résumé ultra-simplifié

Calcule la taille totale des chaînes.
 Alloue la mémoire pour contenir toutes les chaînes concaténées.
 Concatène chaque chaîne.
 Retourne le résultat final.

Analyse ultra-détaillée de array_len()

```
int array_len(char **array)
```

Retourne la taille d'un tableau de chaînes char **.
 Initialisation du compteur

```
int len;
```

```
len = 0;
```

Initialise len = 0 (le nombre d'éléments).
 Comptage des éléments

```
while (array && array[len])
    len++;
```

Pourquoi array && array[len] ?

Empêche un segfault si array est NULL.
 Parcourt chaque élément jusqu'au NULL final.

Retourne la taille

```
return (len);
```

Retourne le nombre total d'éléments dans array.
 Analyse ultra-détaillée de free_array()

```
void free_array(char **array)
```

Libère un tableau char ** (chaque élément + le tableau lui-même).
 Vérification

```
if (!array)
    return;
```

Si array est NULL, on ne fait rien.

Libération des éléments

```
i = 0;
while (array[i])
{
    free(array[i]);
    i++;
}
```

Libère chaque élément un par un.
Libération du tableau

```
free(array);
```

Libère le tableau lui-même.

Analyse ultra-détaillée de process_input()

But de la fonction

Gérer tout le cycle d'exécution d'une commande dans le shell, depuis l'entrée utilisateur jusqu'à l'exécution.

Décryptage ligne par ligne

Déclaration de la fonction

```
void process_input(t_shell *shell)
```

Retourne : void (ne retourne rien).

Prend en entrée :

t_shell *shell → Pointeur vers la structure principale du shell.

Déclaration des variables locales

```
char          *input;
t_token       *tokens;
t_command     *commands;
```

input → Stockera la chaîne de caractères entrée par l'utilisateur.

tokens → Liste chaînée de tokens générée par le tokenizer.

commands → Liste chaînée de commandes extraites des tokens.

Installer les signaux

```
setup_signals();
```

Appelle setup_signals() pour gérer CTRL+C, CTRL+\, CTRL+D.

Lire l'entrée utilisateur

```
input = read_user_input(shell);
if (!input || !shell->running)
    return ;
```

Explication détaillée :

Appelle read_user_input(shell); pour récupérer la commande de l'utilisateur.

Si read_user_input() retourne NULL :

Soit l'utilisateur a appuyé sur CTRL+D → (shell->running est mis à 0 et exit s'affiche).

Soit il y a une erreur mémoire.

Si shell->running == 0, le shell s'arrête immédiatement.

Sinon, on continue avec le traitement de la commande.

Tokeniser l'entrée utilisateur

```
tokens = tokenize_input(input, shell);
if (!tokens)
{
    free(input);
    return ;
}
```

Explication détaillée :

Appelle `tokenize_input(input, shell);`
Analyse l'entrée utilisateur et la découpe en une liste de tokens (arguments, opérateurs, redirections...).

Si `tokens == NULL`, cela signifie que :

- Soit l'entrée est vide après la suppression des espaces.
- Soit une erreur d'allocation mémoire s'est produite.

Dans ce cas, on libère `input` et on quitte immédiatement.

Valider la séquence de tokens

```
if (!validate_token_sequence(tokens))
{
    token_free(&tokens);
    free(input);
    return ;
}
```

Explication détaillée :

Appelle `validate_token_sequence(tokens);`
Vérifie que la commande est syntaxiquement correcte (exemple : pas de pipe mal placé `| |`).

Si la validation échoue (`return 0`) :

- Libère les tokens (`token_free(&tokens);`).
- Libère `input`.

Quitte immédiatement la fonction.

Organiser les commandes

```
commands = organize_commands(tokens, shell);
token_free(&tokens);
if (!commands)
{
    free(input);
    return ;
}
```

Explication détaillée :

Appelle `organize_commands(tokens, shell);`
Convertit la liste des tokens en une liste de `t_command`, qui représente les commandes à exécuter.

Libère les tokens (`token_free(&tokens);`)
Les tokens ne sont plus nécessaires après cette étape.

Si `commands == NULL`, alors :

- Il y a eu une erreur mémoire ou aucune commande valide.

Libère `input` et quitte immédiatement.

Valider les commandes

```
if (validate_commands(commands) < 0)
{
    free_commands(commands);
}
```



```

    free(input);
    return ;
}

```

Explication détaillée :

```

Appelle validate_commands(commands);
    Vérifie que les commandes sont bien formées (ex : un programme
inexistant est une erreur).
Si la validation échoue (return < 0) :
    Libère commands (free_commands(commands));.
    Libère input.
    Quitte immédiatement.

```

Exécuter les commandes

```

execute_commands(shell, commands);

```

Explication détaillée :

```

Appelle execute_commands(shell, commands);
    Exécute la commande via un fork() si nécessaire.
    Vérifie si c'est un builtin (cd, export, etc.) et l'exécute directement
sans fork().

```

Libérer la mémoire

```

free_commands(commands);
free(input);

```

Pourquoi free_commands(commands); ?

La liste commands a été allouée dynamiquement dans organize_commands().

Pourquoi free(input); ?

input est l'entrée de l'utilisateur, et n'est plus utilisée après exécution.

Récapitulatif ultra-simplifié

```

Installe les signaux (setup_signals());.
Lit l'entrée utilisateur (read_user_input());.
Découpe en tokens (tokenize_input());.
Valide la syntaxe des tokens (validate_token_sequence());.
Transforme les tokens en commandes (organize_commands());.
Valide les commandes (validate_commands());.
Exécute les commandes (execute_commands());.
Libère la mémoire (free_commands(); free(input));.

```

Cas où la fonction return en cours de route

Étape Condition d'arrêt

Lecture de l'entrée input == NULL ou shell->running == 0

Tokenisation tokens == NULL

Validation des tokens validate_token_sequence(tokens) == 0

Organisation des commandes commands == NULL

Validation des commandes validate_commands(commands) < 0

```

*****
*****

```